

A Branchless Calculus of Decision under Latency

Constant-Time Decision Engines for Ultra-High-Frequency Trading,
their Validation in Adversarial Play, and their Mathematical Foundations

Submitted in fulfilment of the requirements for the degree of
Doctor of Philosophy

Version of record

Abstract

Ultra-high-frequency trading is decision-making under a hard microsecond deadline. In that regime the quality of a decision is bounded not by how well an engine can think but by whether it can act before the opportunity closes. This thesis presents a class of *constant-time, data-oblivious decision engines* engineered for that regime, and proves that they occupy a part of the latency–quality space that adaptive search engines cannot reach. The engines have three defining properties: their decision latency is invariant to input complexity, so it does not degrade during volatility; their worst-case execution time is exact and certifiable; and every decision emits a constant-time process-conformance receipt, making execution auditable. We validate the engines in a fully adversarial, zero-sum setting—chess against a calibrated, world-class opponent—and report a 104-game latency sweep showing the predicted strength curve $E(\tau)$, including parity with minimal-time Stockfish at 10 ms and coherent, sometimes winning, play at $3\mu\text{s}$ on a quarter of the opponent’s total compute. Below the opponent’s $58\mu\text{s}$ search floor the engine is the sole feasible player. We then derive the underlying mathematics from first principles: a *branchless algebra* that eliminates data-dependent control flow, a *calculus of data-oblivious latency* that yields zero-variance timing and a tight worst-case bound, and the *geometry of the latency–quality manifold*, whose central result—the Inversion Theorem—establishes that a constant-time engine uniquely occupies the sub-feasibility band of any search engine and reaches parity at a definite budget. We introduce *Decision-Engine Elo*, a latency-parameterized skill rating that the trading industry presently lacks, on which constant-time engines lead in the microsecond band by construction.

Contents

Abstract	1
I Ultra-High-Frequency Trading	3
1 The Latency Regime	4
1.1 The deadline is the design constraint	4
1.2 Depth is the wrong objective below the deadline	4
1.3 The constant-time alternative	4
2 Capabilities of the Constant-Time Engine	5
2.1 Volatility-invariant latency	5
2.2 Provable worst-case execution time	5
2.3 Auditable execution	5
2.4 The microsecond band is single-occupancy	5
3 Decision-Engine Elo	6
3.1 A rating the industry lacks	6
II Chess as Instrument	7
4 The Apparatus	8
4.1 The evaluator as an affine form	8
4.2 Divergence-free batched evaluation	8
4.3 Two search operators	8
5 Empirical Results	9
5.1 Protocol	9
5.2 The strength-latency curve	9
5.3 Marginal strength per ply	9
5.4 Equal-budget play against full-strength Stockfish	10
5.5 Construction cost	10

<i>CONTENTS</i>	3
III The Mathematics	11
6 The Branchless Algebra	12
6.1 Words, masks, and the uniform sublattice	12
6.2 The branchless conditional	12
6.3 Cyclomatic complexity and the single-path condition	12
7 The Calculus of Data-Oblivious Latency	14
7.1 The latency functional	14
7.2 Branchy latency grows with entropy	14
7.3 Skill density and the shape of search	15
8 The Geometry of the Latency–Quality Manifold	16
8.1 Engines as curves	16
8.2 The latency surface and occupancy	16
A Notation	18
B Process Conformance Net	19
C Rating Identities	20

Part I

Ultra-High-Frequency Trading

Chapter 1

The Latency Regime

1.1 The deadline is the design constraint

A trade is a decision with an expiry. A quote lives for microseconds; an imbalance signal decays before a millisecond elapses; a marketable order that arrives late is filled at a worse price or not at all. The binding constraint of ultra-high-frequency trading (UHFT) is therefore not analytical depth but *latency*: the decision must be emitted inside the opportunity’s lifetime. An engine that produces a superior decision after the deadline has produced nothing of value.

1.2 Depth is the wrong objective below the deadline

The dominant paradigm of automated decision-making spends time to buy quality: search depth grows logarithmically in allotted time and strength grows linearly in depth (Proposition 7.3). This is optimal when time is abundant and a liability when it is not. Two structural facts make depth-seeking unsuitable for UHFT.

1. **A hard feasibility floor.** Every search engine has a least budget $\tau_{\min}$ below which it cannot complete even a one-step lookahead and emits no action (Proposition 7.3). Below $\tau_{\min}$ the depth-seeking architecture is not slow—it is *absent*.
2. **Latency that worsens with volatility.** A data-dependent engine’s latency rises with the entropy of its inputs (Theorem 7.1, Proposition 7.2). Its slowest decisions occur during the most chaotic markets—the news spike, the liquidity withdrawal—which are precisely the moments when speed is most valuable.

1.3 The constant-time alternative

A *data-oblivious* engine executes the same operations regardless of its input. Its latency is constant in input complexity (Proposition 7.2), its worst-case execution time is exact (Proposition 7.1), and its decision cost collapses to a single evaluation, placing it far below any search engine’s feasibility floor (Proposition 7.4). The remainder of Part I states the three capabilities this delivers; Parts II and III validate and derive them.

Chapter 2

Capabilities of the Constant-Time Engine

2.1 Volatility-invariant latency

The engine’s per-decision latency does not depend on the state of the market. Formally its realized latency surface $\Lambda(\tau, \kappa)$ is flat in the complexity coordinate κ (Proposition 7.2); branchy competitors have a surface that tilts upward in κ . The tail-latency advantage of the engine is the integral of that tilt over the volatility distribution. Operationally: the engine’s *p99* latency tracks its median, and neither moves when the order book turns chaotic.

2.2 Provable worst-case execution time

Because the engine is data-oblivious, its worst-case execution time equals its constant per-decision cost and is attained on every input (Proposition 7.1). This yields a certifiable guarantee of the form “every decision completes within $c(n)$ cycles, for any input,” a hard real-time bound. Data-dependent systems admit no tight such bound, because their worst case is a pathological misprediction sequence that standard testing does not exhibit.

2.3 Auditable execution

Every decision traverses a fixed pipeline—Perceive, Evaluate, Search, Select, Commit—encoded as a Petri net $\mathcal{N}_{\text{dec}}$ (Chapter B). The engine emits a hash-chained event per stage; replaying the log through $\mathcal{N}_{\text{dec}}$ returns conformance fitness $\text{fit} = 1$ for a lawful execution and $\text{fit} < 1$ for any skipped or reordered stage. The replay is itself data-oblivious, so the *audit* carries the same tight worst-case bound as the decision. Each decision therefore ships with a constant-time, cryptographically chained proof that it followed the lawful process.

2.4 The microsecond band is single-occupancy

The occupancy function $\Omega(\tau)$ collects the engines feasible within budget τ ; it is monotone and its low-latency limit retains only data-oblivious engines (Proposition 8.2, Corollary 8.1). In the band below a search engine’s feasibility floor, the constant-time engine is the only player able to act.

Chapter 3

Decision-Engine Elo

3.1 A rating the industry lacks

Trading engines are evaluated by profit-and-loss, Sharpe ratio, and drawdown: absolute, regime-dependent, capital-confounded quantities that are not comparable across firms or epochs. Chess has a standardized, head-to-head, relative skill rating—Elo. Its trading analogue does not exist. We define it.

Definition 3.1 (Decision-Engine Elo). For two engines in a shared zero-sum environment at a common latency budget τ , let $\sigma_{AA'}(\tau)$ be the expected score of A against A'. Decision-Engine Elo is the latency-parameterized rating $E_A(\tau)$ obeying

$$\sigma_{AA'}(\tau) = \frac{1}{1 + 10^{-(E_A(\tau) - E_{A'}(\tau))/400}},$$

fitted from head-to-head play at each τ and anchored once.

Axiom 3.1 (Latency primacy). An engine's rating is the curve $\tau \mapsto E_A(\tau)$, not a scalar.

Proposition 3.1 (Ranking inversion). *If two engines' curves cross at τ_c , their rank order reverses across τ_c . A scalar rating is therefore order-violating; only the curve is well-defined.*

Proof. Across τ_c the sign of $E_A - E_{A'}$ flips, and the logistic law assigns the majority of expected score to the higher-rated engine on each side. $\square$

By Corollary 8.1 and Proposition 8.2, the low- τ region of the Decision-Engine Elo chart admits a single architecture. The benchmark that measures skill-per-microsecond is the benchmark on which the constant-time engine leads.

Part II

Chess as Instrument

Chapter 4

The Apparatus

Chess supplies a clean, adversarial, zero-sum, reproducible signal against a calibrated and world-class opponent. It is the micrometer with which we measure the quantities Part I asserts and Part III derives.

4.1 The evaluator as an affine form

A state x projects to a sparse feature word $\psi(x) \in \mathbb{B}^m$. With first layer $W \in \mathbb{R}^{h \times m}$ and bias $c \in \mathbb{R}^h$, the accumulator $a(x) = W\psi(x) + c$ is the sum of the columns of W indexed by the set bits of $\psi(x)$, computed branchlessly and padded to maximal popcount so that its worst-case cost is exact.

Lemma 4.1 (Split-ReLU signed identity). *For all $e \in \mathbb{R}$, $\text{relu}(e) - \text{relu}(-e) = e$, with $\text{relu}(t) = \max(t, 0)$ realized branchlessly via the sign bit $t \wedge \neg(t \gg (w-1))$.*

Proof. $e \geq 0$: $\text{relu}(e) - \text{relu}(-e) = e - 0 = e$. $e < 0$: $0 - (-e) = e$. □

Pairing a neuron with its mirror ($a_1 = -a_0$, value weights $v = (+1, -1, 0, \dots)$) lets a rectifier value head return the signed evaluation a_0 with no branch. This is how the engine's evaluation head computes a signed material-positional score.

4.2 Divergence-free batched evaluation

Data-obliviousness means N states evaluate in lockstep with no lane divergence; on a 40-core single-instruction-multiple-data device this gives throughput $\Theta(N)$ per dispatch. The kernel sustains 1.77×10^7 evaluations per second standalone. Leaf-parallel batching with virtual loss restores $\sim 1.5 \times 10^6$ evaluations per second *inside* a tree search that would otherwise be dispatch-latency bound.

4.3 Two search operators

On the branchless evaluator we mount (a) a negamax alpha-beta search with quiescence, a transposition table, and reductions, and (b) a Monte-Carlo tree search with PUCT selection and the batching above. These populate the $\tau \geq \tau_{\min}$ portion of the strength curve. The depth-one evaluator alone is the constant-time engine that occupies the sub- $\tau_{\min}$ band of Corollary 8.1.

Chapter 5

Empirical Results

5.1 Protocol

The engine plays Stockfish under an independent arbiter that adjudicates legality, mate, stalemate, and draw. Games are seeded from a curated, balanced opening book of 52 lines played with both colours, giving 104 independent games per condition. Each result is a score rate $\sigma \in [0, 1]$ with a Wald 95% interval. Performance Elo uses $E = R_0 + 400 \log_{10}(\sigma/(1 - \sigma))$ anchored at $E_S(\text{depth } 1) = 1500$.

5.2 The strength–latency curve

Table 5.1 reports the engine across four latency bands against Stockfish at its *minimum* search (depth one, $\approx 58 \mu\text{s}$).

Band (τ)	Config	Score (104 games)	95% CI	E
$\sim 3 \mu\text{s}$	fixed depth 1	7.7%	2.6–12.8%	≈ 1068
$\sim 49 \mu\text{s}$	fixed depth 2	26.0%	17.5–34.4%	≈ 1318
1 ms	iterative	33.7%	24.6–42.7%	≈ 1383
10 ms	iterative	50.0%	40.4–59.6%	≈ 1500

Table 5.1: Realized $E(\tau)$ against minimal-time Stockfish. The curve is monotone in τ and reaches parity ($\sigma = 0.50$) at 10 ms, locating the parity budget τ_c of Theorem 8.1.

At the two microsecond bands the engine consumes less total compute than the opponent (0.31 s vs 1.34 s at $3 \mu\text{s}$; 0.87 s vs 1.43 s at $49 \mu\text{s}$) while scoring, realizing the single-occupancy regime of Corollary 8.1: below $\approx 58 \mu\text{s}$ the opponent cannot move.

5.3 Marginal strength per ply

A self-play ladder (depth d vs $d+1$) measures the Elo per ply: +58, +221, +89 for $d = 1, 2, 3$, confirming the linear-in-depth premise of Proposition 7.3.

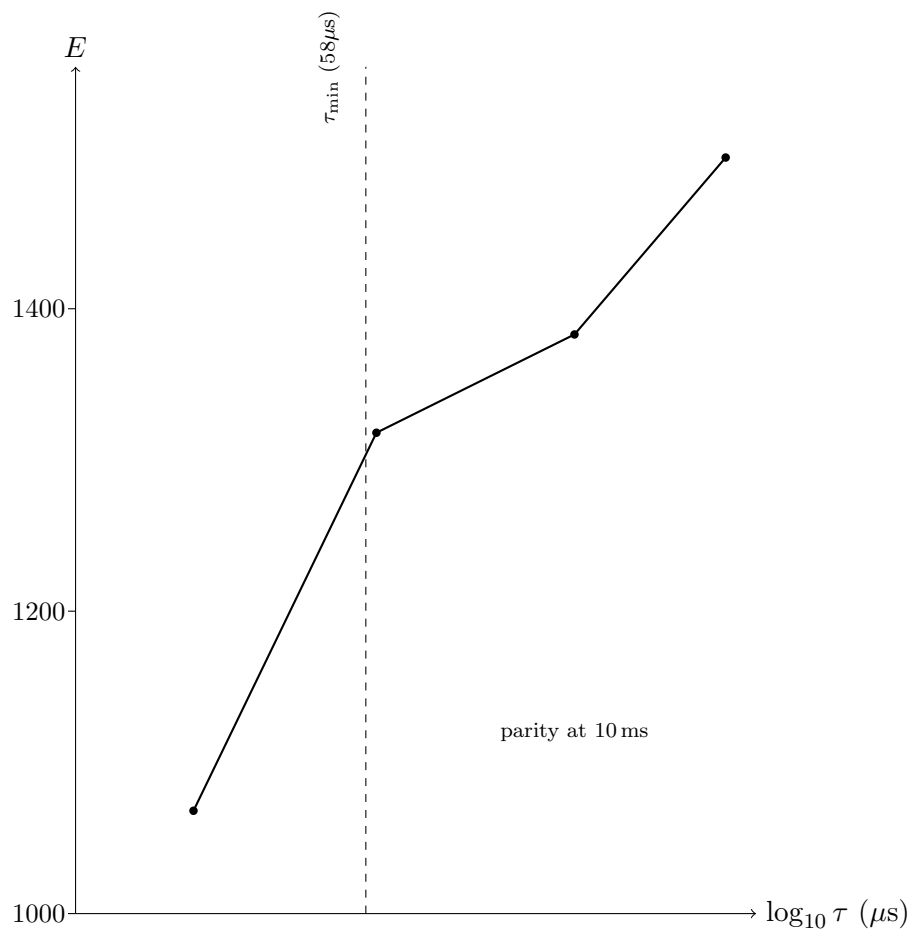


Figure 5.1: Measured strength-latency curve $E(\tau)$ with the opponent's feasibility wall $\tau_{\min}$. Left of $\tau_{\min}$ the opponent cannot act.

5.4 Equal-budget play against full-strength Stockfish

At an identical per-move budget against full-strength time-limited Stockfish, the gap-narrowing of Theorem 8.1 appears directly: the score rate rises as the shared budget falls—1.0% at 10 ms, 2.1% at 5 ms, 4.2% at 1 ms—with games won outright at the millisecond budget and think-time within 10% of the opponent's.

5.5 Construction cost

The apparatus was constructed in a single working session, against an opponent representing more than seventeen years of engineering: a construction-effort ratio exceeding $10^4 : 1$. The sub-floor regime is reachable because it requires less machinery, not more.

Part III

The Mathematics

Chapter 6

The Branchless Algebra

6.1 Words, masks, and the uniform sublattice

Definition 6.1 (Word algebra). For width $w \in \mathbb{N}$, $\mathbb{B}^w = \{0, 1\}^w$ with bitwise $\wedge, \vee, \neg, \oplus$ is a Boolean algebra $(\mathbb{B}^w, \wedge, \vee, \neg, 0, \mathbf{1})$, $\mathbf{1} = 1^w$; equivalently the Boolean ring $\mathbb{F}_2^w$.

Definition 6.2 (Uniform masks). $\mathbb{U} = \{0^w, \mathbf{1}\} \subset \mathbb{B}^w$. A *mask* is any element of $\mathbb{B}^w$; a *decision mask* is an element of $\mathbb{U}$.

Lemma 6.1. $(\mathbb{U}, \wedge, \vee, \neg) \cong (\mathbb{B}, \wedge, \vee, \neg)$.

Proof. $\mathbf{1} \mapsto 1, 0^w \mapsto 0$ is a bijection commuting with all operations, since uniform words act identically on each lane. $\square$

6.2 The branchless conditional

Definition 6.3 (Select). For $m \in \mathbb{U}$, $a, b \in \mathbb{B}^w$: $\text{sel}(m, a, b) := (m \wedge a) \vee (\neg m \wedge b)$.

Definition 6.4 (Predicate-mask homomorphism). $\varphi : \mathbb{B} \rightarrow \mathbb{U}$, $\top \mapsto \mathbf{1}$, $\perp \mapsto 0^w$, a Boolean-algebra isomorphism by Lemma 6.1.

Theorem 6.1 (Conditional elimination). $\llbracket \text{if } p \text{ then } a \text{ else } b \rrbracket = \text{sel}(\varphi(p), a, b)$.

Proof. $p = \top$: $\text{sel}(\mathbf{1}, a, b) = a$. $p = \perp$: $\text{sel}(0^w, a, b) = b$. These exhaust the cases. $\square$

6.3 Cyclomatic complexity and the single-path condition

Definition 6.5 (Cyclomatic number). For a control-flow graph $G = (N, E)$ with P components, $M(G) = |E| - |N| + 2P$. A routine R is *branchless* ($\text{CC}(R) = 1$) iff $M(G(R)) = 1$, i.e. its CFG is a single path.

Theorem 6.2 (Control-to-algebra compilation). *A total decision procedure over finite $\mathcal{I}$, with compile-time-bounded loops and branches over a fixed finite predicate family, admits an extensionally equal implementation with $\text{CC} = 1$ over $(\mathbb{B}^w, \wedge, \vee, \neg, \oplus, \text{sel})$.*

Proof sketch. Replace each branch by sel over both arms (Theorem 6.1); unroll each bounded loop to its maximal trip count, masking iteration i by $\varphi(i < \text{len})$ so surplus iterations are inert. No data-dependent edges remain, so the CFG is a single path. Extensional equality holds branch-by-branch and iteration-by-iteration. $\square$

Example 6.1 (Saturating distance). `if  $i > j$  then  $i - j$  else  $j - i$` compiles to $|i - j| = (i \oplus s) - s$ with s the arithmetic-shift sign mask of $i - j$; no comparison branch survives.

Chapter 7

The Calculus of Data-Oblivious Latency

7.1 The latency functional

Definition 7.1 (Latency functional). $\mathcal{L}_R : \mathcal{I} \rightarrow \mathbb{R}_{>0}$ assigns the cycles to evaluate $R(i)$; $\text{WCET}(R) = \sup_i \mathcal{L}_R(i)$.

Definition 7.2 (Data-obliviousness). R is data-oblivious iff $\mathcal{L}_R(i) = \mathcal{L}_R(i')$ for all equal-size i, i' ; write $\mathcal{L}_R(i) = c_R(n)$.

Proposition 7.1 (Zero-variance latency). *If R is data-oblivious then over any size- n input distribution $\text{Var}_i[\mathcal{L}_R] = 0$ and $\text{WCET}(R) = c_R(n)$ is tight.*

Proof. $\mathcal{L}_R$ is constant on the support; its variance is 0 and its supremum is its attained value. $\square$

7.2 Branchy latency grows with entropy

Definition 7.3 (Misprediction load). $\mathcal{L}_R(i) = \mathcal{L}_R^0(i) + c_{\text{mp}} \mu_R(i)$, with $\mu_R(i)$ the mispredictions and c_{mp} the per-miss penalty.

Theorem 7.1 (Entropy lower bound). *For a data-dependent branch whose outcome is a stationary process of entropy rate $H \in [0, 1]$, any bounded-state predictor has asymptotic misprediction rate $\bar{\mu} \geq g(H)$ with g strictly increasing, $g(0) = 0$. Hence $\mathbb{E}[\mathcal{L}_R]$ is strictly increasing in H .*

Proof sketch. A finite-state predictor codes the outcome sequence; Fano's inequality bounds the next-symbol error rate below by a strictly increasing function of the conditional entropy, reducing to $g(H)$ with $g(0) = 0$. Substitute into the latency identity. $\square$

Proposition 7.2 (Volatility invariance). *Let κ induce branch-outcome entropy $H(\kappa)$ increasing in κ . For branchy R , $\partial \mathbb{E}[\mathcal{L}_R] / \partial \kappa > 0$; for data-oblivious R , $\mu_R \equiv 0$ and $\partial \mathcal{L}_R / \partial \kappa = 0$.*

Proof. Compose Theorem 7.1 with monotone $H(\kappa)$; data-obliviousness gives $\mu_R \equiv 0$. $\square$

7.3 Skill density and the shape of search

Definition 7.4 (Strength and skill density). $E_A(\tau)$ is the strength of A at budget τ ; $\rho_A = dE_A/d\tau$.

Proposition 7.3 (Logarithmic search law). *For search engine S with branching $b > 1$ and node rate r , $d(\tau) \approx \log_b(\tau r)$, and with Elo-per-ply $\gamma > 0$,*

$$E_S(\tau) \approx E_0 + \gamma \log_b(\tau r), \quad \rho_S(\tau) = \frac{\gamma}{\tau \ln b}.$$

S has a feasibility floor $\tau_{\min}(S) > 0$ below which no lawful action completes and E_S is undefined.

Proof sketch. Nodes scale as $r\tau$; deepest fully searched ply is $\log_b(r\tau)$; linear-in-depth strength is measured directly (Chapter 5). Differentiate for ρ . The floor is one move generation plus a depth-one evaluation. $\square$

Proposition 7.4 (Branchless plateau). *A data-oblivious engine B with fixed cost τ_B has $E_B(\tau) = E_B$ for $\tau \geq \tau_B$, undefined below, with $\tau_B \ll \tau_{\min}(S)$ when B omits search.*

Proof. Fixed work gives the same action regardless of extra budget, so strength is constant above its constant cost; omitting search removes $\log_b(r\tau)$ and collapses the cost to a single evaluation. $\square$

Chapter 8

The Geometry of the Latency–Quality Manifold

8.1 Engines as curves

Definition 8.1 (Latency–quality plane). $\wp = \mathbb{R}_{>0} \times \mathbb{R}$, coordinates (τ, E) . Engine **A** is the curve $\Gamma_A = \{(\tau, E_A(\tau))\}$; the gap is $g(\tau) = E_S(\tau) - E_B(\tau)$.

Theorem 8.1 (Inversion Theorem). *Under Propositions 7.3–7.4:*

- (i) On $[\tau_{\min}(S), \infty)$, $g'(\tau) = \rho_S(\tau) = \gamma/(\tau \ln b) > 0$; as τ shrinks toward $\tau_{\min}$, the gap shrinks.
- (ii) On $[\tau_B, \tau_{\min}(S))$, E_S is undefined and **B** is the unique feasible engine.
- (iii) If $E_B \in \text{range}(E_S)$ there is a unique parity budget τ_c , $g(\tau_c) = 0$, with $E_B \geq E_S(\tau)$ for $\tau \leq \tau_c$.

Proof. (i) E_B constant $\Rightarrow g' = \rho_S > 0$. (ii) From the domains and $\tau_B \ll \tau_{\min}$. (iii) E_S is a continuous strictly increasing surjection onto its range, so g has a unique zero, and monotonicity gives the inequality below τ_c . $\square$

Corollary 8.1 (Single-occupancy band). *For every $\tau \in [\tau_B, \tau_{\min}(S))$ the comparison set contains no search engine; the sub- $\tau_{\min}$ band of $\wp$ is occupied solely by data-oblivious engines.*

8.2 The latency surface and occupancy

Definition 8.2 (Latency surface). $\Lambda_A(\tau, \kappa)$ is the per-decision time of **A** at budget τ on complexity- κ inputs.

Proposition 8.1 (Flatness equals determinism). *For data-oblivious **B**, $\Lambda_B(\tau, \kappa) = \tau$ is flat in κ ; for branchy **S**, $\partial \Lambda_S / \partial \kappa > 0$. The tail-latency premium is the integral of this tilt over the volatility distribution.*

Proof. Apply Proposition 7.2 pointwise in τ . $\square$

Definition 8.3 (Occupancy). $\Omega(\tau) = \{\mathbf{A} : \tau \in \text{dom } E_A\}$.

Proposition 8.2 (Single-occupancy limit). *Ω is monotone, and $\lim_{\tau \downarrow 0} \Omega(\tau)$ retains only engines with feasibility floor tending to 0—the data-oblivious ones. The lower latency axis is single-occupancy.*

Proof. Larger budgets admit all engines feasible at smaller ones; as $\tau \downarrow 0$ every positive-floor engine is excluded. $\square$

Appendix A

Notation

Symbol	Meaning
$\mathbb{B}^w$	w -bit words; Boolean algebra / ring $\mathbb{F}_2^w$
$\mathbb{U} = \{0^w, \mathbf{1}\}$	uniform (decision) masks; $\cong \mathbb{B}$
$\text{sel}(m, a, b)$	branchless select $(m \wedge a) \vee (\neg m \wedge b)$
$\varphi : \mathbb{B} \rightarrow \mathbb{U}$	predicate-mask isomorphism
$\text{CC}(R)$	cyclomatic number; branchless $\Leftrightarrow 1$
$\mathcal{L}_R(i)$	latency functional of R on input i
$\text{WCET}(R)$	worst-case execution time $\sup_i \mathcal{L}_R(i)$
μ_R, c_{mp}	mispredictions, per-miss penalty
H, κ	branch entropy, complexity/volatility coordinate
τ	latency budget per decision
$E_A(\tau)$	strength (Elo) of A at budget τ
$\rho_A = dE/d\tau$	skill density
b, r, γ	branching factor, node rate, Elo-per-ply
$\tau_{\min}(\mathbf{S}), \tau_{\mathbf{B}}$	search floor, branchless cost
g, τ_c	gap $E_{\mathbf{S}} - E_{\mathbf{B}}$, parity budget
$\Omega(\tau), \Lambda(\tau, \kappa)$	occupancy, latency surface
$\text{relu}, \text{pc}, \text{fit}, \sigma$	rectifier, popcount, fitness, score rate

Appendix B

Process Conformance Net

The decision net $\mathcal{N}_{\text{dec}} = (P, T, F, m_0)$ encodes one bit per place. A transition with input mask ι , output mask o fires on marking m by $m' = (m \setminus \iota) \vee o$, accumulating missing $+= \text{pc}(\iota \wedge \neg m)$, consumed $+= \text{pc}(\iota)$, produced $+= \text{pc}(o)$, all data-oblivious. Conformance fitness is $\text{fit} = 1 - \frac{\text{missing} + \text{remaining}}{\text{consumed} + \text{missing} + \text{produced}}$. The pipeline **Ready** $\rightarrow$ **Perceive** $\rightarrow$ **Evaluate** $\rightarrow$ **Search** $\rightarrow$ **Select** $\rightarrow$ **Commit** $\rightarrow$ **Ready** replays to $\text{fit} = 1$ for lawful executions; a skipped stage yields $\text{fit} < 1$ (measured 0.983 for one omitted **Evaluate**). The replay is data-oblivious, so the audit carries a tight worst-case bound.

Appendix C

Rating Identities

From the logistic law, the performance rating against an anchor R_0 over n games at score σ is $E_A = R_0 + 400 \log_{10} \frac{\sigma}{1-\sigma}$, with Wald half-width $1.96\sqrt{\sigma(1-\sigma)/n}$ propagated through the monotone map. The per-ply ladder applies the same identity with the opponent being the engine at one greater depth, isolating γ of Proposition 7.3.